

LAPORAN PERCOBAAN PRAKTIKUM JOBSHEET 6

SORTING (BUBBLE, SELECTION, DAN INSERTION SORT)

Nama : Jonathan Abdiel Haryono

Absen : 14

Kelas : TI 1G

1. Praktikum 1 : Mengimplementasikan Sorting Menggunakan Object

Sorting14.java

```
package Praktikum05;

public class Sorting14 {

    int []data;
    int jumData;

    Sorting14 (int Data[], int jmlDat) {
        jumData = jmlDat;
        data = new int[jmlDat];

        for (int i = 0; i < jumData; i++) {
            data[i] = Data[i];
        }
    }

    void bubbleSort() {
        int temp = 0;
        for (int i = 0; i < jumData-1; i++) {
            for (int j = 1; j < jumData-i; j++) {
                if (data[j-1] > data[j]) {
                    temp = data[j];
                    data[j] = data [j-1];
                    data[j-1] = temp;
                }
            }
        }
    }

    void tampil() {
        for (int i = 0; i < jumData; i++) {
            System.out.print(data[i] + " ");
        }
        System.out.println();
    }

    void SelectionSort() {
        for (int i = 0; i < jumData-1; i++) {
            int min = i;
```

```

        for (int j = i+1; j < jumData; j++) {
            if (data[j] < data[min]) {
                min = j;
            }
        }

        int temp = data[i];
        data[i] = data[min];
        data[min] = temp;

    }
}

void insertionSort() {
    for (int i = 1; i < jumData; i++) {
        int temp = data[i];
        int j = i-1;

        while (j >= 0 && data[j] > temp) {
            data[j+1] = data[j];
            j--;
        }
        data[j+1] = temp;
    }
}
}
}

```

SortingMain14.java

```
package Praktikum05;
```

```

public class SortingMain14 {
    public static void main(String[] args) {
        int a[] = {20, 10, 2, 7, 12};
        int b[] = {30, 20, 2, 8, 14};
        int c[] = {40, 10, 4, 9, 3};

        Sorting14 dataurut1 = new Sorting14(a, a.length);
        Sorting14 dataurut2 = new Sorting14(b, b.length);
        Sorting14 dataurut3 = new Sorting14(c, c.length);

        System.out.println("Data awal 1");
        dataurut1.tampil();
        dataurut1.bubbleSort();

        System.out.println("Data sudah diurutkan dengan BUBBLE SORT (ASC)");
        dataurut1.tampil();

        System.out.println("Data awal 2");
        dataurut2.tampil();
        dataurut2.SelectionSort();

        System.out.println("Data sudah diurutkan dengan SELECTION SORT (ASC)");
    }
}

```

```

dataurut2.tampil();

System.out.println("Data awal 3");
dataurut3.tampil();
dataurut3.SelectionSort();

System.out.println("Data sudah diurutkan dengan INSERTION SORT (ASC)");
dataurut3.tampil();
}
}

```

a. SORTING – BUBBLE SORT

Verifikasi Hasil Percobaan

```

Data awal 1
20 10 2 7 12
Data sudah diurutkan dengan BUBBLE SORT (ASC)
2 7 10 12 20

```

b. SORTING – SELECTION SORT

Verifikasi Hasil Percobaan

```

Data awal 2
30 20 2 8 14
Data sudah diurutkan dengan SELECTION SORT (ASC)
2 8 14 20 30

```

c. SORTING – INSERTION SORT

Verifikasi Hasil Percobaan

```

Data awal 3
40 10 4 9 3
Data sudah diurutkan dengan INSERTION SORT (ASC)
3 4 9 10 40

```

Pertanyaan

1. Jelaskan fungsi kode program berikut

```
if (data[j-1]>data[j]){  
    temp=data[j];  
    data[j]=data[j-1];  
    data[j-1]=temp;  
}
```

2. Tunjukkan kode program yang merupakan algoritma pencarian nilai minimum pada selection sort!
3. Pada Insertion sort , jelaskan maksud dari kondisi pada perulangan
`while (j>=0 && data[j]>temp)`
4. Pada Insertion sort, apakah tujuan dari perintah `data[j+1]= data[j];`

Jawaban

1. Kode tersebut berfungsi untuk melakukan swapping. Cara kerjanya adalah sebagai berikut.
Pertama, program akan mengecek apakah data[j-1] lebih besar dari data[j]. Apabila benar, program akan mengalihkan sementara nilai data[j] ke dalam temp (temporary), kemudian mengubah nilai data[j] dengan nilai data[j-1], dan mengakhirinya dengan mengubah nilai data[j-1] dengan nilai temp yang mana merupakan nilai dari data[j] sebelumnya. Sehingga, terjadi pertukaran data (swapping) antara data[j] dengan data[j-1].
2.

```
for (int j = i+1; j < jumData; j++) {  
    if (data[j] < data[min]) {  
        min = j;  
    }  
}
```
3. Kondisi tersebut adalah kondisi ketika $j \geq 0$ **dan** $data[j] > temp$. Sebelum membaca kondisi, ada baiknya untuk memperhatikan code sebelum dan setelahnya. Tertulis “int temp = data[i];” dan “int j = i-1;” sebelum while. Variabel “temp” mereferensi pada data yang ke-i (dimulai dari index 1) dan variabel j bernilai i-1 (i akan terus bertambah seiring perulangan for). Di dalam perulangan while, tertulis “data[j+1] = data[j];” dan “j--;”. Potongan code “data[j+1] = data[j];” berarti mengubah nilai data[j+1] (yakni data saat ini) dengan data[j] (data sebelumnya) dan bagian terpentingnya adalah potongan code “j--;”, yakni berarti mengurangi nilai j secara terus menerus selama while terpenuhi. Nah, $j \geq 0$ menjadi batasan agar j yang berkurang secara terus menerus tadi dapat berhenti pada index ke-0. Selanjutnya, $data[j] > temp$ berarti mengecek apakah ada data-data sebelumnya yang

nilainya lebih besar daripada nilai temp (yakni data saat ini) untuk mempertimbangkan melakukan swapping. Sehingga, ketika kedua kondisi tersebut terpenuhi, yakni ketika j yang terus menerus berkurang tidak melewati batasannya (yakni 0) dan nilai data yang ke-j (bernilai i-1 di awal) lebih besar daripada nilai temp (data saat ini), nilai data saat ini (data[j+1]) akan diubah menjadi data yang ke-j (data[j]) dan nilai j akan berkurang sebanyak 1.

4. Seperti yang pernah saya singgung pada soal nomor 3, perintah tersebut bertujuan untuk mengubah nilai data saat ini (data[j+1]) menjadi data yang ke-j (data[j]), yakni nilai yang lebih besar daripada data saat ini (data[j+1]). Kemudian, setelah while, akan diakhiri dengan "data[j+1] = temp" yang artinya mengubah nilai data yang ke-j+1 (yakni data yang memenuhi tadi) dengan nilai temp, yakni nilai data yang ke-i (data saat ini).

2. Praktikum 2 : Sorting Menggunakan Array of Object

Mahasiswa14.java

```
package Praktikum05;
```

```
public class Mahasiswa14 {  
    String nim, nama, kelas;  
    double ipk;
```

```
    //Konstruktor default  
    Mahasiswa14() {
```

```
    }
```

```
    // Konstruktor berparameter (dibuat ada yang nama var parameter inputnya sama ada yang tidak)
```

```
    Mahasiswa14(String nm, String name, String kls, double ip) {  
        nim = nm;  
        nama = name;  
        ipk = ip;  
        kelas = kls;  
    }
```

```
    void tampilInformasi() {  
        System.out.println("Nama    : " + nama);  
        System.out.println("NIM    : " + nim);  
        System.out.println("Kelas : " + kelas);  
        System.out.println("IPK    : " + ipk);  
    }  
}
```

MahasiswaBerprestasi14.java

```
package Praktikum05;

public class MahasiswaBerprestasi14 {
    Mahasiswa14[] listMhs = new Mahasiswa14[5];
    int idx;

    void tambah (Mahasiswa14 m) {
        if (idx < listMhs.length) {
            listMhs[idx] = m;
            idx++;
        }
        else {
            System.out.println("Data sudah penuh");
        }
    }

    void tampil() {
        for (Mahasiswa14 m : listMhs) {
            m.tampilInformasi();
            System.out.println("-----");
        }
    }

    void bubbleSort() {
        for (int i = 0; i < listMhs.length-1; i++) {
            for (int j = 1; j < listMhs.length-i; j++) {
                if (listMhs[j].ipk > listMhs[j-1].ipk) {
                    Mahasiswa14 tmp = listMhs[j];
                    listMhs[j] = listMhs[j-1];
                    listMhs[j-1] = tmp;
                }
            }
        }
    }
}
```

MahasiswaDemo14.java

```
package Praktikum05;

public class MahasiswaDemo14 {
    public static void main(String[] args) {
        MahasiswaBerprestasi14 list = new MahasiswaBerprestasi14();
        Mahasiswa14 m1 = new Mahasiswa14("123", "Zidan", "2A", 3.2);
        Mahasiswa14 m2 = new Mahasiswa14("124", "Ayu", "2A", 3.5);
        Mahasiswa14 m3 = new Mahasiswa14("125", "Sofi", "2A", 3.1);
        Mahasiswa14 m4 = new Mahasiswa14("126", "Sita", "2A", 3.9);
        Mahasiswa14 m5 = new Mahasiswa14("127", "Miki", "2A", 3.7);

        list.tambah(m1);
        list.tambah(m2);
        list.tambah(m3);
        list.tambah(m4);
    }
}
```

```

        list.tambah(m5);

        System.out.println("Data mahasiswa sebelum sorting: ");
        list.tampil();

        System.out.println("\nData Mahasiswa setelah sorting berdasarkan IPK (DESC) : ");
        list.bubbleSort();
        list.tampil();
    }
}

```

Verifikasi Hasil Percobaan

Data mahasiswa sebelum sorting: Nama : Zidan NIM : 123 Kelas : 2A IPK : 3.2 ----- Nama : Ayu NIM : 124 Kelas : 2A IPK : 3.5 ----- Nama : Sofi NIM : 125 Kelas : 2A IPK : 3.1 ----- Nama : Sita NIM : 126 Kelas : 2A IPK : 3.9 ----- Nama : Miki NIM : 127 Kelas : 2A IPK : 3.7 -----	Data Mahasiswa setelah sorting berdasarkan IPK (DESC) : Nama : Sita NIM : 126 Kelas : 2A IPK : 3.9 ----- Nama : Miki NIM : 127 Kelas : 2A IPK : 3.7 ----- Nama : Ayu NIM : 124 Kelas : 2A IPK : 3.5 ----- Nama : Zidan NIM : 123 Kelas : 2A IPK : 3.2 ----- Nama : Sofi NIM : 125 Kelas : 2A IPK : 3.1 -----
---	---

Pertanyaan

- Perhatikan perulangan di dalam bubbleSort() di bawah ini:

```

for (int i=0; i<listMhs.length-1; i++){
    for (int j=1; j<listMhs.length-i; j++){

```

 - Mengapa syarat dari perulangan i adalah `i<listMhs.length-1` ?
 - Mengapa syarat dari perulangan j adalah `j<listMhs.length-i` ?
 - Jika banyak data di dalam listMhs adalah 50, maka berapakah perulangan i akan berlangsung? Dan ada berapa **Tahap** bubble sort yang ditempuh?
- Modifikasi program diatas dimana data mahasiswa bersifat dinamis (input dari keyboard) yang terdiri dari nim, nama, kelas, dan ipk!

Jawaban

- A. Perulangan outer bersyarat `i < listMhs.length-1` karena perulangan outer selalu terjadi sebanyak `n-1` kali (`n` = jumlah data).

B. Perulangan inner bersyarat $j < \text{listMhs.length} - i$ karena setiap perulangan inner selalu terjadi sebanyak $n - i$ kali (n = jumlah data dan i = index perulangan outer).

C. Dengan jumlah data sebanyak 50, perulangan i (dengan rumus $n - 1$) akan terjadi sebanyak 49 kali yang artinya ada 49 tahap bubble sort yang akan ditempuh.

2. **Mahasiswa14.java**

```
package Praktikum05;

public class Mahasiswa14 {
    String nim, nama, kelas;
    double ipk;

    //Konstruktor default
    Mahasiswa14() {

    }

    // Konstruktor berparameter (dibuat ada yang nama var parameter inputnya sama ada yang tidak)
    /* Mahasiswa14(String nm, String name, String kls, double ip) {
        nim = nm;
        nama = name;
        ipk = ip;
        kelas = kls;
    }
    */

    void tampilInformasi() {
        System.out.println("Nama    : " + nama);
        System.out.println("NIM    : " + nim);
        System.out.println("Kelas : " + kelas);
        System.out.println("IPK    : " + ipk);
    }
}
```

MahasiswaBerprestasi14.java

```
package Praktikum05;

public class MahasiswaBerprestasi14 {
    Mahasiswa14[] listMhs = new Mahasiswa14[5];
    int idx;

    void tambah (Mahasiswa14 m) {
        if (idx < listMhs.length) {
            listMhs[idx] = m;
            idx++;
        }
    }
}
```



```

        else {
            System.out.println("Data sudah penuh");
        }
    }

    void tampil() {
        for (int i = 0; i < idx; i++) {
            listMhs[i].tampilInformasi();
            System.out.println("-----");
        }
    }

    void bubbleSort() {
        // Ganti listMhs.length menjadi idx
        for (int i = 0; i < idx - 1; i++) {
            for (int j = 1; j < idx - i; j++) {
                if (listMhs[j].ipk > listMhs[j-1].ipk) {
                    Mahasiswa14 tmp = listMhs[j];
                    listMhs[j] = listMhs[j-1];
                    listMhs[j-1] = tmp;
                }
            }
        }
    }
}

```

MahasiswaDemo14.java

```

package Praktikum05;
import java.util.Scanner;

public class MahasiswaDemo14 {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Masukkan jumlah mahasiswa : ");
        int jumlahMhs = sc.nextInt();

        MahasiswaBerprestasi14 list = new MahasiswaBerprestasi14();
        Mahasiswa14 mhs[] = new Mahasiswa14[jumlahMhs];

        for (int i = 0; i < jumlahMhs; i++) {

            sc.nextLine();

            mhs[i] = new Mahasiswa14();

            System.out.println("-----");
            System.out.println("Mahasiswa ke-" + (i+1));
            System.out.print("NIM   : ");
            mhs[i].nim = sc.nextLine();
            System.out.print("Nama   : ");
            mhs[i].nama = sc.nextLine();
        }
    }
}

```

```

        System.out.print("Kelas : ");
        mhs[i].kelas = sc.nextLine();
        System.out.print("IPK  : ");
        mhs[i].ipk = sc.nextDouble();

        list.tambah(mhs[i]);
    }

    System.out.println("-----");

    System.out.println("\nData mahasiswa sebelum sorting:");

    for (int i = 0; i < jumlahMhs; i++) {
        System.out.println("-----");
        System.out.println("Mahasiswa ke-" + (i+1));
        System.out.println("NIM  : " + mhs[i].nim);
        System.out.println("Nama  : " + mhs[i].nama);
        System.out.println("Kelas : " + mhs[i].kelas);
        System.out.println("IPK   : " + mhs[i].ipk);
    }

    System.out.println("\nData Mahasiswa setelah sorting berdasarkan IPK (DESC) : ");
    System.out.println("-----");

    list.bubbleSort();

    list.tampil();

    /*
    Mahasiswa14 m1 = new Mahasiswa14("123", "Zidan", "2A", 3.2);
    Mahasiswa14 m2 = new Mahasiswa14("124", "Ayu", "2A", 3.5);
    Mahasiswa14 m3 = new Mahasiswa14("125", "Sofi", "2A", 3.1);
    Mahasiswa14 m4 = new Mahasiswa14("126", "Sita", "2A", 3.9);
    Mahasiswa14 m5 = new Mahasiswa14("127", "Miki", "2A", 3.7);

    list.tambah(m1);
    list.tambah(m2);
    list.tambah(m3);
    list.tambah(m4);
    list.tambah(m5);

    */
}
}

```

3. Praktikum 3 : Mengurutkan Data Mahasiswa Berdasarkan IPK (Selection Sort)

Verifikasi Hasil Percobaan

```
Masukkan jumlah mahasiswa : 5
-----
Mahasiswa ke-1
NIM   : 123
Nama  : Ali
Kelas : 2B
IPK   : 3,9
-----
Mahasiswa ke-2
NIM   : 124
Nama  : Ila
Kelas : 2B
IPK   : 3,1
-----
Mahasiswa ke-3
NIM   : 125
Nama  : Agus
Kelas : 2B
IPK   : 3,6
-----
Mahasiswa ke-4
NIM   : 126
Nama  : Tika
Kelas : 2B
IPK   : 3,3
-----
Mahasiswa ke-5
NIM   : 127
Nama  : Udin
Kelas : 2B
IPK   : 3,2
-----

Data yang sudah terurut menggunakan SELECTION SORT (ASC)
-----
Nama   : Ila
NIM    : 124
Kelas : 2B
IPK    : 3.1
-----
Nama   : Udin
NIM    : 127
Kelas : 2B
IPK    : 3.2
-----
Nama   : Tika
NIM    : 126
Kelas : 2B
IPK    : 3.3
-----
Nama   : Agus
NIM    : 125
Kelas : 2B
IPK    : 3.6
-----
Nama   : Ali
NIM    : 123
Kelas : 2B
IPK    : 3.9
-----
```

Pertanyaan

Di dalam method selection sort, terdapat baris program seperti di bawah ini:

```
int idxMin=i;
for (int j=i+1; j<listMhs.length; j++){
    if (listMhs[j].ipk<listMhs[idxMin].ipk){
        idxMin=j;
    }
}
```

Untuk apakah proses tersebut, jelaskan!

Jawaban

1. Proses tersebut digunakan untuk mencari nilai paling minimum dari index ke-i+1 hingga panjang listMhs dengan membandingkan nilai listMhs[idxMin].ipk satu per satu dengan nilai listMhs[j].ipk. Jika terpenuhi, index nilai terkecil tadi akan dimasukkan dalam idxMin.

4. Praktikum 4 : Mengurutkan Data Mahasiswa Berdasarkan IPK Menggunakan Insertion Sort

Verifikasi Hasil Percobaan

```
Masukkan jumlah mahasiswa : 5
-----
Mahasiswa ke-1
NIM : 111
Nama : Ayu
Kelas : 2C
IPK : 3,7
-----
Mahasiswa ke-2
NIM : 222
Nama : Dika
Kelas : 2C
IPK : 3,0
-----
Mahasiswa ke-3
NIM : 333
Nama : Ila
Kelas : 2C
IPK : 3,8
-----
Mahasiswa ke-4
NIM : 444
Nama : Susi
Kelas : 2C
IPK : 3,1
-----
Mahasiswa ke-5
NIM : 555
Nama : Yayuk
Kelas : 2C
IPK : 3,4
-----

Data yang sudah terurut menggunakan INSERTION SORT (ASC)
-----
Nama : Dika
NIM : 222
Kelas : 2C
IPK : 3.0
-----
Nama : Susi
NIM : 444
Kelas : 2C
IPK : 3.1
-----
Nama : Yayuk
NIM : 555
Kelas : 2C
IPK : 3.4
-----
Nama : Ayu
NIM : 111
Kelas : 2C
IPK : 3.7
-----
Nama : Ila
NIM : 333
Kelas : 2C
IPK : 3.8
-----
```

Pertanyaan

Ubahlah fungsi pada InsertionSort sehingga fungsi ini dapat melaksanakan proses sorting dengan cara descending.

Jawaban

```
void insertionSort() { // modif : asc to desc
    for (int i = 1; i < listMhs.length; i++) {
        Mahasiswa14 temp = listMhs[i];
        int j = i;
        while (j > 0 && listMhs[j-1].ipk > temp.ipk) {
            listMhs[j] = listMhs[j-1];
            j--;
        }
        listMhs[j] = temp;
    }
}
```

Mengubah > menjadi <